OPERATIONS MANUAL

EDITION 2.0 · OCTOBER 2026

WHATSAPP SALES & OPERATIONS CONSOLE

ChatCloser

Documentation, feature reference and operating guide

BY THIRTEEN

VERSION	PLATFORMS	STATUS	LICENCE
2.0.0	Windows desktop · Server	Release documentation	MIT

DOCUMENT CONTROL

About this document

Everything an operator, administrator or developer needs to install ChatCloser 2.0, run it day to day, and understand exactly what the software will and will not do.

PRODUCT

ChatCloser 2.0.0

PUBLISHER

Thirteen

DOCUMENT EDITION

1.0 — October 2026

APPLIES TO

Desktop (Windows x64) and server deployments

AUDIENCE

Owners, managers, staff, administrators, developers

LICENCE

MIT

How this document is organised

PART	WHAT IT COVERS	READ IT WHEN
I — Foundations	What the product is, how it is built, how to install it and sign in.	First day.
II — Operations	Every page of the console, what it shows and how to use it.	Daily work.
III — The assistant	How replies are produced, which tools exist, how to configure models.	Before going live.
IV — Channels & money	WhatsApp connection, payment providers and verification.	When connecting accounts.
V — Administration	Security, backups, configuration, deployment, troubleshooting.	Running it for a team.

Conventions

INTERFACE LABELS

Bold text names a button, page or field exactly as it appears in the console, for example **Settings & Backups** or **Create backup**.

COMMANDS

Commands are shown in monospace blocks. Run them from the project directory unless a step says otherwise. Windows PowerShell examples are used by default.

PLACEHOLDERS

Angle brackets such as `<business-id>` stand for a value you supply. Never type the brackets.

GUARANTEES

Statements marked with a gold rule are invariants enforced in code, not settings. They are listed in §2 and §30.

FAST PATH

Installing today? Read §3, then §4, then the setup checklist in §27. Running a team? Add §20 and §30. Connecting WhatsApp and payments? §28 and §29.

CONTENTS

What is inside

Thirty-nine sections in five parts. Every entry is a live link in this PDF; the sidebar bookmark panel mirrors the same structure.

PART I — FOUNDATIONS

01 Product overview →
02 Architecture and trust model →
03 Requirements and installation →
04 First run and sign-in →
05 Interface map →

PART II — OPERATIONS

06 Overview →
07 AI Command Center →
08 Customer Inbox →
09 Leads and CRM →
10 Customers →
11 Products and services →
12 Inventory →
13 Orders and bookings →
14 Quotations, invoices and expenses →
15 Payments →
16 Automation Studio →
17 Business Intelligence →
18 Knowledge Base →
19 Tasks, reminders and notifications →
20 Team and permissions →
21 Security and audit logs →
22 Settings and backups →
23 Account →

PART III — THE ASSISTANT

24 How the assistant works →
25 Providers, models and cost control →
26 Tool reference →
27 Teaching it your business →

PART IV — CHANNELS AND MONEY

28 WhatsApp →
29 Payment providers in depth →

PART V — ADMINISTRATION

30 Security model →
31 Data, backup and migration →
32 Configuration reference →
33 Deployment →
34 The desktop application →
35 Troubleshooting →
36 Limitations and non-goals →
37 Upgrading from ChatCloser 1.x →
38 Glossary →
39 Support, licence and credits →

PART I

Foundations

What ChatCloser is, the principles that shape every screen, and the ten minutes it takes to have it running on a real machine.

1. Product overview
2. Architecture and trust model
3. Requirements and installation
4. First run and sign-in
5. Interface map

SECTION 01

Product overview

ChatCloser is a self-hosted WhatsApp sales and operations console. It answers customers, quotes from your catalogue, records leads, takes confirmed orders, collects verified payments and keeps a complete audit trail — without sending your data to a third-party CRM.

What it does

A customer messages your business on WhatsApp. The assistant answers in English or Nepali, looks up real products and prices, checks stock and delivery charges, and — when the customer is ready — prepares a priced order summary. Nothing is charged and no order exists until the customer explicitly confirms that summary. From there the order moves through your own workflow: processing, dispatch, delivery, payment, invoice, follow-up. Staff can take over any conversation at any moment, and every action is recorded against the person or tool that performed it.

19	17	8	15	129
CONSOLE PAGES	ASSISTANT TOOLS	AI PROVIDERS	INDUSTRY TEMPLATES	AUTOMATED TESTS

Who it is for

Small and mid-sized businesses that already sell through WhatsApp and want the repetitive part — questions, prices, availability, order taking, payment links, follow-ups — handled consistently, with a human able to step in. The industry templates ship working module sets and extra intake fields for:

SALES
Retail & wholesale
E-commerce & social commerce

FOOD & HOSPITALITY
Restaurants, cafés & delivery
Hotels & hospitality

PROPERTY & PROJECTS
Construction & building materials
Real estate & property services

SERVICES
Professional services & agencies
Repair & maintenance
Salons & personal services

CARE & LEARNING
Clinics & appointments
Education & training

SUPPLY & TRAVEL
Manufacturing & distribution
Agriculture & supplies
Travel & tourism
Custom workflows

The five guarantees

These are enforced in code and covered by tests. They are the reason the console can be trusted with money and customer conversations.

GUARANTEE	WHAT IT MEANS IN PRACTICE
Prices come from the catalogue	The assistant may only quote figures returned by a catalogue tool. If an item is not in **Products & services**, it cannot be sold by the assistant, and prices cannot be invented or remembered from the chat.
Orders need explicit confirmation	A draft is a priced summary, not an order. `confirm_customer_order` fails unless the customer said yes to those exact items, quantities and total. If a price changed in between, the assistant must ask again.

GUARANTEE	WHAT IT MEANS IN PRACTICE
Paid means the provider said so	Payment status is written only from a verified provider response or an approved manual receipt. A customer claiming “I already paid” changes nothing.
The model proposes, the backend disposes	The assistant never executes code. It requests a named tool with JSON arguments; the backend validates the schema, checks permissions and approvals, runs the operation and returns a structured result.
Everything is scoped and audited	Every query is filtered by the business taken from the signed-in session — never from a request parameter. Privileged actions write an audit entry with actor, action and details.

Two editions, one codebase

EDITION	BEST FOR	HOW IT RUNS
Desktop	A single shop or office computer that stays on. No Node.js, no terminal, no server administration.	Signed Windows installer or portable executable. The window is the console; the tray icon keeps WhatsApp connected after the window is closed.
Server	An always-on machine, a VPS, or several staff using the console from their own computers.	Node.js process behind a TLS reverse proxy. Runs headless as a service; the same console, same database format.

> **WHATSAPP TRANSPORT**
>
> ChatCloser connects through the WhatsApp Web protocol (Baileys), the same mechanism as WhatsApp Web, not the official Business API. It works with an ordinary number, but WhatsApp's terms do not guarantee it, and numbers that send unsolicited bulk messages can be restricted. Use a dedicated business number, message only people who contacted you, and keep volumes human. See §28.

SECTION 02

Architecture and trust model

A single Node.js process, one SQLite database and a strict boundary between what the language model may ask for and what the system will actually do.

Components

Channels
WhatsApp via Baileys. Inbound messages, media metadata, delivery receipts.

→

Conversation core
Threads, messages, assignment, takeover, internal notes.

→

Assistant engine
Language detection, prompt assembly, model call, tool loop, reply validation.

→

Domain services
Catalogue, inventory, orders, payments, quotes, workflows, reports.

→

SQLite
WAL mode, migrations, one file to back up.

The console is an Express application served by the same process. It reads and writes through the same services, so a staff member clicking **Confirm** and the assistant calling `confirm_customer_order` pass through identical validation. Payment providers and outgoing webhooks are the only outbound network calls besides the AI provider.

LAYER	LOCATION	RESPONSIBILITY
Core	`src/core`	Configuration, database connection, RBAC, audit log, validation, error types.

LAYER	LOCATION	RESPONSIBILITY
Database	`src/db`	Connection pragmas, migration runner, numbered migrations 001–006.
Services	`src/services`	Business rules: customers, catalogue, inventory, orders, quotations, payments, workflows, knowledge, reports, backups.
Assistant	`src/services/ai`	Engine, language detection, provider adapters, 17 validated tools.
HTTP API	`src/server`	Control API on loopback, session store, security headers, webhooks.
Console	`dashboard`	Server-rendered pages, forms, CSRF, permissions per route.
Desktop shell	`electron-app`	Window, tray, first-run dialog, data directory, packaging.

Life of an inbound message

1. **Receive.** The WhatsApp socket emits a message event. Duplicates are ignored by provider message id; media is downloaded only when needed.
2. **Record.** The thread is created or found, the message stored with direction, sender and timestamp, and the conversation marked unread for staff.
3. **Route.** If a human has taken over, the assistant stays silent. If the assistant is paused, disabled or the message is from a group, it is queued for a person instead.
4. **Assemble.** Business profile, enabled modules, catalogue excerpts, knowledge search results and the last twelve messages are assembled into the prompt.
5. **Reason.** The model replies with text, or requests a tool call. Tool arguments are validated against a JSON schema before anything runs.
6. **Act.** The backend checks the business's permissions and tool settings, executes the operation, and returns a structured result. Read tools run freely; writes are constrained by the invariants in §1.
7. **Validate and send.** The reply is checked (length, empty output, leaked internals), stored, and sent through a per-chat and global rate limiter.

Trust boundaries

#	BOUNDARY	CONTROL
1	**Customer → assistant**	Customer text is untrusted input. It is never executed, never interpolated into SQL, and always escaped when displayed.
2	**Model → backend**	Only named tools with schema-validated arguments. No arbitrary code, no shell, no SQL from the model.
3	**Backend → database**	Parameterised statements, tenant id from the session, foreign keys enforced.
4	**Browser → console**	Session cookie (HttpOnly, SameSite=Lax), CSRF token on every state-changing form, CSP with nonces, frame denial.
5	**Provider → payments**	Signature or server-side verification before a payment is marked paid; provider events are idempotent.
6	**Operator → secrets**	API keys and payment credentials encrypted at rest with AES-256-GCM under a key held outside the database.

WHY ONE PROCESS

A single process keeps the desktop edition genuinely simple: one executable, one database, one log. The control API (default port 3002) is bound to loopback and exists for local integrations and health checks; it is not a public interface. The console (default port 3001) is also loopback-only until you deliberately change `DASHBOARD_HOST`.

SECTION 03

Requirements and installation

Choose the desktop edition for one computer, or the server edition for an always-on deployment. Both run the same application and store data in the same format.

Requirements

REQUIREMENT	DESKTOP	SERVER
Operating system	Windows 10 or 11, 64-bit	Linux, macOS or Windows
Runtime	Bundled — nothing to install	Node.js 22.13 or newer (24 LTS recommended)
Disk	About 250 MB installed, plus the database	About 300 MB, plus the database and backups
Memory	2 GB available	1 GB available
Network	Outbound HTTPS to your AI provider and WhatsApp	Same, plus inbound HTTPS if staff connect remotely
Ports	3001 and 3002 free on loopback	3001 (console) and 3002 (control), or your own values
Phone	A WhatsApp number for the business, able to scan a QR code	Same

Desktop installation

1. **Run the installer.** `ChatCloser-Setup-2.0.0-x64.exe` installs for the current user into `%LOCALAPPDATA%\Programs\ChatCloser` and creates desktop and Start Menu shortcuts. It does not require administrator rights.
2. **Pass SmartScreen.** The build is not code-signed, so Windows shows “Windows protected your PC”. Choose **More info** → **Run anyway**. Verify the SHA-256 of the file you received against the published value if you obtained it through a channel you do not fully trust.
3. **Launch.** The window opens on the sign-in page within a few seconds; a tray icon appears at the same time.
4. **Note the credentials.** On first run ChatCloser creates its own encryption key, database and owner account, and shows the generated password once in a **Welcome to ChatCloser** dialog. The same text is written to `%APPDATA%\ChatCloser\first-run-credentials.txt`.
5. **Sign in and change the password.** **Account** → **Change password**, then delete the credentials file.

Prefer not to install? The portable build, `ChatCloser-Portable-2.0.0-x64.exe`, unpacks to a temporary folder and runs immediately. Data still lives in `%APPDATA%\ChatCloser`, so the portable and installed editions share a workspace on the same machine.

Server installation

```
git clone <repository> chatcloser
cd chatcloser
npm ci

# create the environment file and generate real secrets
cp .env.example .env
node -e "console.log('ENCRYPTION_KEY=' + require('crypto').randomBytes(32).toString('base64'))"
node -e "console.log('SESSION_SECRET=' + require('crypto').randomBytes(32).toString('base64'))"

# edit .env: set the two keys above, ADMIN_USERNAME, ADMIN_PASSWORD,
# BUSINESS_NAME, and INDUSTRY_TEMPLATE

npm run migrate
npm start
```

`npm start` runs the application server: WhatsApp client, assistant, scheduler and console. The console is at `http://127.0.0.1:3001`. On first start the owner account is created; if `ADMIN_PASSWORD` is empty a strong password is generated and printed once to the console.

> **SECRETS ARE CHECKED AT STARTUP**
>
> In server mode ChatCloser refuses to start with placeholder or missing `ENCRYPTION_KEY` / `SESSION_SECRET` values. The encryption key protects stored provider credentials and encrypted backups — if you lose it, those cannot be recovered. Keep a copy in a password manager, separate from the database.

Building the installers yourself

```
npm ci
npm test                      # 129 tests, no network required
npm run desktop:dist:win      # writes to electron-app/dist
```

The build stages the backend, its production dependencies and the Electron shell into an asar archive, then produces `ChatCloser-Setup-2.0.0-x64.exe` and `ChatCloser-Portable-2.0.0-x64.exe`. The packaging step fails deliberately if the runtime dependency list in `electron-app/package.json` drifts from the root manifest, because a missing dependency only shows up as a crash on a customer's machine.

COMMAND	WHAT IT DOES
`npm start`	Full application: bot, assistant, scheduler and console.
`npm run dashboard`	Console only, for inspecting an existing database.
`npm run bot`	WhatsApp client and assistant without the console.
`npm run migrate`	Applies pending migrations and exits.
`npm test`	Runs the automated suite against temporary databases.
`npm run desktop`	Runs the desktop shell in development.

SECTION 04

First run and sign-in

The first launch provisions the workspace. There is no demo data: an empty dashboard is the correct starting state, and the checklist tells you what to fill in.

What happens on first launch

STEP	RESULT
Database created	SQLite file with all migrations applied, in WAL mode.
Encryption key created	Stored in `secrets.json` in the data directory; used for provider credentials and backups.
Business provisioned	One business record with your chosen industry template, its modules and extra intake fields.
Owner account created	Username `admin` unless configured otherwise, with a generated password shown once.
WhatsApp started	The client connects and waits for a QR scan; the status becomes *waiting for QR* within a few seconds.

Signing in

Open the console at `http://127.0.0.1:3001` (the desktop window already points there). Enter the username and password. Sessions last twelve hours and are stored in the database, so a restart does not sign you out. **Sign out** in the sidebar ends the session immediately.

> IF YOU LOSE THE PASSWORD
>
> Passwords are stored only as scrypt hashes and cannot be read back. If you are still signed in, use **Account → Change password**. If you are locked out and the installation has no other owner, stop the app and rename the data folder — a new workspace with fresh credentials is created and the old database remains in the renamed folder for recovery.

Roles at a glance

ROLE	BUILT FOR	CAN DO
Owner	The person accountable for the business.	Everything, including settings, team, integrations and backups. Always includes permissions added in future versions.
Manager	Day-to-day supervisor.	Conversations, leads, customers, catalogue view, orders (create, edit, cancel), bookings, quotes, invoices view, payments view, reports, AI view, knowledge view, workflows view, team view.
Staff	Sales and support.	Inbox replies, leads, customers view, orders view, bookings view, catalogue view, inventory view, quotes view, invoices view, knowledge view.
Accountant	Bookkeeping.	Payments manage and refund, invoices edit, quotes and orders view, customer export, reports.
Viewer	Owner or auditor who should not change anything.	Read-only access to every operational area; no settings, no payments actions.

Roles are bundles; individual permissions can be granted or revoked per user without inventing a new role. The full matrix is in §20.

SECTION 05

Interface map

The sidebar is the whole application. Items you do not have permission for are hidden, and the server checks the same permission again on every request.

The sidebar, top to bottom

PAGE	PERMISSION	PURPOSE
Overview	`dashboard.view`	Today at a glance: KPIs, 14-day sales, setup checklist, recent assistant conversations.
AI Command Center	`ai.view`	What the assistant has been doing: recent threads, model usage, available tools.
Customer Inbox	`conversations.view`	Every WhatsApp thread; take over, reply, assign, leave internal notes.
Leads & CRM	`leads.view`	Enquiries captured by the assistant or entered by staff, with status tracking.
Orders & Bookings	`orders.view`	Confirmed orders, pending drafts awaiting customer confirmation, appointments.
Products & Services	`catalog.view`	The catalogue: the only source of names and prices the assistant may use.
Inventory	`inventory.view`	Stock levels, movements, reservations, low-stock thresholds, valuation.
Quotations & Invoices	`quotes.view`	Numbered quotations, invoices with printable documents, expenses.
Payments	`payments.view`	Payment attempts, provider status, manual approvals, reconciliation.
Customers	`customers.view`	Customer profiles, tags, order and payment history, CSV export.
Automation Studio	`workflows.view`	Trigger → condition → action rules, with an execution log.
Business Intelligence	`reports.view`	Revenue, funnel, product mix, forecast, inventory risk, accounting summary.
AI Configuration	`ai.view`	Provider, model, key, persona, temperature, limits, tool switches.
Knowledge Base	`knowledge.view`	Documents the assistant may quote: policies, delivery, FAQs, service areas.
Integrations	`integrations.view`	WhatsApp connection, payment providers, outgoing webhooks.
Team & Permissions	`team.view`	Staff accounts, roles, per-permission overrides.
Security & Audit Logs	`audit.view`	Who did what, when, and to which record.
Settings & Backups	`settings.view`	Company profile, tax, delivery, hours, modules, encrypted backups.
Account	any signed-in user	Your name, username, password.

Conventions used throughout

- **Cards** group one task. The card title is the action, the small text underneath explains the rule.

- **Status pills** use the same vocabulary everywhere: an order state, a payment state, a thread state. The words in the pill are the words the API and the audit log use.
- **Money** is shown in the workspace currency with two decimals, formatted for the locale. Changing the currency does not convert existing records.
- **Timestamps** are shown in the workspace timezone, which defaults to `Asia/Kathmandu`.
- **Destructive actions** are forms with a confirmation step; nothing is deleted by a stray click.

> TIP
>
> The dashboard is server-rendered and works without JavaScript for every core action. The only scripted behaviour is progressive enhancement — the WhatsApp QR refresh, chart drawing and auto-submitting status selects — so a blocked script cannot lock you out of an operation.

PART II

Operations

Every page of the console, in the order it appears in the sidebar: what it shows, what you can do there, and the rules that page enforces.

1. Overview
2. AI Command Center
3. Customer Inbox
4. Leads and CRM
5. Customers
6. Products and services
7. Inventory
8. Orders and bookings
9. Quotations, invoices and expenses
10. Payments
11. Automation Studio
12. Business Intelligence
13. Knowledge Base
14. Tasks, reminders and notifications
15. Team and permissions
16. Security and audit logs
17. Settings and backups
18. Account

SECTION 06

Overview

The morning screen: money collected, work waiting, and the five setup steps between an empty workspace and a live assistant.

The twelve counters

Every number is computed from your database at page load. Nothing is cached and nothing is pre-filled, so an empty workspace honestly shows zeros.

COUNTER	MEANING
Revenue (paid)	Sum of verified payments, not order totals. Unpaid and unverified amounts are excluded.
Paid orders	Orders whose payment status is paid.
Pending orders	Orders that exist but are not yet paid. Highlighted when non-zero.
New leads	Leads still in the *new* state.
Lead conversion	Leads that reached *booked* or *closed*, as a percentage of all leads.
Awaiting reply	Unread customer messages across all threads.
Low-stock alerts	Items at or below their reorder level.
Unverified payments	Payments in *requires review* — provider said something ambiguous, or a receipt is waiting for approval.
Failed workflows	Workflow runs that ended in error in the last day.
Awaiting approval	Approvals queued by tools or workflows that require a human decision.
Open tasks	Tasks not yet completed or cancelled.
AI runs (30 days)	Assistant invocations, a proxy for workload and cost.

Sales — last 14 days

A table of paid revenue per day with an inline bar for visual comparison. It uses the same verified-payment rule as the revenue counter, so the chart and the number never disagree.

Getting started

Five rows, each linking straight to the page that completes it: company profile and industry template, AI configuration, catalogue, knowledge base, WhatsApp connection. The checklist is a convenience, not a gate — nothing is blocked, but an assistant without a catalogue cannot quote and an assistant without a key cannot reply.

Recent conversations

The last threads the assistant handled, with customer, outcome and a link into the conversation. Use it as a quick sanity check that replies sound like your business.

> DAILY ROUTINE
>
> Read the warning-tinted counters first — pending orders, awaiting reply, low stock, unverified payments, failed workflows, awaiting approval. Those six are the only counters that represent work someone must do. Everything else is context.

SECTION 07

AI Command Center

An oversight view of the assistant: what it answered, which tools it used, and how much model capacity that consumed.

What the page shows

BLOCK	CONTENT
Model usage	Runs and token consumption for the period, per provider and model, so cost surprises are visible before the invoice.
Recent conversations	Threads the assistant handled, with the last message and outcome (replied, escalated, handed over, failed).
Tools the assistant may use	The full tool list with what each does. Switches live on **AI Configuration**; this page is the reference.

Reading the tool list

Tools are grouped by risk. Read-only tools (`search_products` , `check_stock` , `calculate_quote` , `get_order_status` , `get_payment_status` , `get_delivery_options` , `get_product_details`) can run on any turn without consequence. Write tools change data and are the ones worth reviewing when a reply looks wrong: `create_order_draft` , `submit_order_for_confirmation` , `confirm_customer_order` , `cancel_order` , `create_booking` , `create_support_ticket` , `schedule_follow_up` , `capture_lead` , `request_human_takeover` .

> OVERSIGHT, NOT CONFIGURATION
>
> Nothing on this page changes behaviour. Provider, model, key, persona, temperature and per-tool switches are all on **AI Configuration**. Keeping the two separate means an operator can inspect the assistant without being one keystroke from reconfiguring it.

SECTION 08

Customer Inbox

Every WhatsApp conversation in one list, with the ability to stop the assistant, assign the thread and leave notes for colleagues.

The list

Columns: last message time, customer, preview, status, AI mode, priority, assignee and unread count. Filter by free-text search (name, number or message text) and by status. Threads are ordered by most recent activity.

FIELD	VALUES
Status	`open` , `pending` , `closed` — your team's own triage state.
AI mode	`auto` while the assistant replies; `human` after a takeover.
Priority	`low` , `normal` , `high` , `urgent` .
Unread	Increments on each inbound message, cleared when the thread is opened.

A conversation

The detail page shows the message history as bubbles with timestamps, the customer's number, status, AI mode, priority and assignee, and three management cards.

ACTION	EFFECT
Take over	Switches the thread to human mode and stops automatic replies immediately. The optional summary is stored with the handover so the next person has context.
Resume AI	Returns the thread to automatic replies. Use it when the customer's issue is resolved or routine again.
Assign	Records a username as responsible. Assignment is a label for people; it does not change permissions.
Internal notes	Timestamped notes with author. Notes are never sent to the customer and never enter the assistant's prompt.

> THERE IS NO COMPOSE BOX — BY DESIGN
>
> ChatCloser is not a WhatsApp Web replacement. After a takeover you reply from WhatsApp itself, signed in as the linked business number (the phone that scanned the QR, or WhatsApp Web on a linked device). The console records the thread, the handover and the notes, and the assistant stays silent until you press **Resume AI**. This avoids two applications fighting over one session and keeps the customer on the number they already know.

SECTION 09

Leads and CRM

Enquiries that are not yet orders. The assistant records them when it has enough to be useful; staff can move them through a simple pipeline.

Lead records

Each lead keeps the customer name, the service or item requested, location, urgency, the originating chat and a status. The table is ordered newest first and shows four counters above it: new, contacted, booked and closed.

STATUS	USE IT WHEN
new	Captured, nobody has followed up yet.
contacted	Someone has spoken to the customer.
booked	The enquiry produced an appointment or a confirmed order.
closed	Finished, one way or the other. Keep it for reporting rather than deleting.

Working the list

- Change a status from the select in the row — it saves immediately, with the signed-in user recorded as the actor.
- **Export CSV** produces the current list for a spreadsheet or a handover.
- The **Customer Inbox** link on each row jumps to the conversation the lead came from.

> LEADS VERSUS CUSTOMERS
>
> A lead is an opportunity; a customer is a person you have a relationship with. The assistant creates the customer record when a conversation starts, and captures a lead when there is a concrete need worth following up. Both are linked, so a lead always has a phone number behind it.

SECTION 10

Customers

One profile per phone number, with the full commercial history: orders, payments, messages and leads.

The list

Search by name or phone, sorted by most recent activity. Each row links to the profile. Totals such as lifetime value are refreshed from orders and payments, not typed in by hand.

A profile

BLOCK	CONTENT
Identity	Name, phone number, email if known, preferred language, consent flags.
Tags	Free-form labels such as *wholesale*, *VIP*, *Kathmandu*, used for filtering and for workflow conditions.
Orders	Every order with state, total and payment status; opens the order.
Payments	Provider, reference, amount and verified status.
Messages	The conversation history, including assistant replies.
Leads	Enquiries captured from this customer.

Housekeeping

- **Export CSV** includes the customer list for accounting or migration. Exporting requires the `customers.export` permission.
- Deleting a customer is a soft delete: the record is hidden from lists but orders and payments keep their history, as bookkeeping requires.
- Duplicate phone numbers are detected before insert; the API exposes a duplicates report for cleaning up legacy data.

SECTION 11

Products and services

The catalogue is the assistant's only source of names and prices. If it is not here, the assistant cannot sell it.

Adding an item

The **Add an item** card takes a name, SKU, price, category and description, and two switches: track inventory and enabled. Saving creates the product for the current business and writes an audit entry.

FIELD	NOTES
Name	Exactly how the assistant should say it. Customers will see this wording.
SKU	Optional but recommended; searchable, and useful for wholesale buyers.
Price	In the workspace currency, tax handling per §22. Zero is allowed for free items.
Category	Groups items on the page and in reports.

FIELD	NOTES
Description	Short factual text. The assistant quotes it; avoid marketing claims you cannot honour.
Track inventory	When on, orders reserve and consume stock and low-stock alerts apply.
Enabled	Disabled items stay in the database but are invisible to the assistant and cannot be ordered.

Items beyond the form

The console form covers the common case. The service layer and the JSON API also support variants (size, colour, weight), per-item tax rates, units, attributes and barcodes — useful for wholesale and pharmacy-style catalogues. Products are never hard-deleted while referenced by an order; disabling is the safe way to retire an item.

> THE PRICE RULE
>
> Editing a price does not retroactively change confirmed orders. A draft that was prepared at the old price fails to confirm after a price change, and the assistant is required to show the customer the new total and ask again. That failure is intentional.

SECTION 12

Inventory

Stock levels, movements, reservations and the alerts that stop you promising what you do not have.

What the page shows

BLOCK	CONTENT
Stock levels	On-hand quantity per item and variant, reorder level, and a low-stock flag.
Recent movements	Every change with reason, quantity, reference (order, return, adjustment) and author.
Valuation	On-hand quantity multiplied by cost or price, depending on configuration, for a quick capital figure.

How stock moves

Stock is not edited by typing a new number. It changes through movements, each with a reason, so the history always explains the balance:

- **Reserve** when an order is confirmed — available quantity drops, on-hand stays until dispatch.
- **Consume** when goods leave; **release** when an order is cancelled or a reservation expires.
- **Receive** for purchases or returns, **adjust** for stock counts and damage.

Movements are created by orders, returns, workflows and the JSON API (`POST /api/inventory/movement`). The console is deliberately read-only: a stock number is a financial statement, and a free-text field invites silent errors.

> BEFORE PROMISING A DELIVERY DATE
>
> The assistant calls `check_stock` before confirming availability. That tool reads the same reserved-versus-on-hand figures shown here, so the answer in the chat and the page agree. If you sell the last unit in the shop, record the movement — the assistant will otherwise keep offering it.

SECTION 13

Orders and bookings

The heart of the system: an eleven-state order lifecycle, drafts that are not orders, and appointments that share the same customer record.

Three tables on one page

- **Orders** — confirmed and later states, with number, customer, total, payment status, state and links.
- **Pending order drafts** — summaries waiting for the customer's yes. This is the queue a manager watches.
- **Bookings** — appointments and service visits with their own state.

The order lifecycle

draft
Being prepared. Not an order.

→

awaiting_confirmation
Summary sent; waiting for the customer.

→

confirmed
Customer approved; stock reserved.

→

processing
Being picked, prepared or scheduled.

→

dispatched
On the way.

→

delivered
Received by the customer.

Side exits are explicit: `cancelled`, `rejected`, `payment_failed`, `refunded` and `returned`. Only the transitions listed in the table below are accepted; anything else is refused with a conflict error rather than silently applied.

FROM	ALLOWED NEXT STATES
draft	awaiting_confirmation, cancelled
awaiting_confirmation	confirmed, rejected, cancelled, draft
confirmed	processing, cancelled, payment_failed
processing	dispatched, cancelled
dispatched	delivered, returned
delivered	returned, refunded
payment_failed	confirmed, cancelled
cancelled, rejected, refunded, returned	terminal

Confirming a draft

Confirm on a pending draft turns it into a real order. Before that happens the backend re-reads every price from the catalogue; if anything changed, the draft is rejected and the customer must approve the new total. Stock is reserved in the same transaction. A confirmation is idempotent per draft, so a double click cannot create two orders.

Bookings

STATE	MEANING
scheduled	Agreed but not yet confirmed by staff.
confirmed	Staff accepted the slot.
in_progress	Service under way.

STATE	MEANING
completed	Delivered and closed.
cancelled, no_show	Ended without delivery; kept for reporting.

EXPORTS

Export CSV on the orders table includes number, customer, dates, totals, payment status and state — the same fields the reports use, so a spreadsheet and the dashboard reconcile.

SECTION 14

Quotations, invoices and expenses

Numbered commercial documents generated from real order data, printable from the console, and expenses for the accounting summary.

Numbering

Quotations are numbered `QUO-YYYY-0001` and invoices `INV-YYYY-0001`, with the sequence restarting each calendar year and scoped per business. Numbers are assigned inside the same transaction that creates the document, so two staff members cannot receive the same number.

Quotations

A quotation freezes a set of lines and totals so the customer has something stable to consider. It can be created from an existing order (`createQuotationFromOrder`), by a workflow action, or through `POST /api/quotations`. The page lists them with number, customer, date, total and status, and links to the printable document.

Invoices

Invoices are generated from confirmed orders and carry the order reference, customer, tax breakdown, delivery charge and amount paid. The **Print** link opens a document-only page — no sidebar, no navigation — ready for the browser's print dialogue or “Save as PDF”.

Expenses

Simple records of money out: category, amount, who was paid, a note and the date. They feed the accounting summary in §17 so gross revenue and net position are not confused. Expenses are created through `POST /api/expenses`; the page lists the most recent ones.

WHY CREATION LIVES IN THE API

Document creation is a commercial act that deserves a deliberate interface. In this release the console is the viewer and printer; creation happens from orders, workflows or the loopback API. This keeps a stray click from issuing an invoice, and it means every document has a traceable origin.

SECTION 15

Payments

Money is only ever “paid” when a provider says so. This page shows every attempt, its verified state and the actions available to staff.

Statuses

STATUS	MEANING
pending	Created, customer has not completed the flow.
succeeded	Provider confirmed the charge. Order payment status becomes paid.
failed	Provider refused or the attempt errored.
cancelled	Customer abandoned or the link expired.
refunded	Refund recorded and reconciled.
requires_review	Ambiguous outcome, signature failure or a manual receipt awaiting approval. Never treated as paid.

What staff can do

- **Verify** re-queries the provider for a pending or ambiguous payment and applies the verified result.
- **Approve** a manual payment (cash, bank transfer) after checking the receipt. Approval records the approver and the note.
- **Reconcile** an order to compare its total against the sum of verified payments and flag shortfalls.
- Open the order from any row to continue the workflow.

What the page never allows

- Marking a payment paid by typing — only verification or approval can do that.
- Trusting a customer's screenshot; the provider reference is checked server-side.
- Double-counting a provider webhook; events are recorded by id and applied once.

> **UNVERIFIED MEANS UNVERIFIED**
>
> `requires_review` is not a soft “probably paid”. Orders in that state do not count as revenue, do not trigger paid-order workflows and do not release goods automatically. Someone must look.

SECTION 16

Automation Studio

Rules made of data, not code: a trigger, optional conditions and a short list of actions, with every run logged.

Triggers

TRIGGER	FIRES WHEN
Incoming message	A customer message arrives on a connected channel.

TRIGGER	FIRES WHEN
New lead	The assistant or a staff member captures a lead.
Order created	A confirmed order exists (after customer approval).
Order confirmed	The transition into `confirmed` succeeds.
Payment verified	A payment becomes `succeeded` with a provider-backed result.
Stock threshold reached	An item falls to or below its reorder level.
Appointment approaching	A booking is within its reminder window.
Delivery status changed	An order moves to dispatched, delivered or returned.
Scheduled time	A time-based rule, evaluated by the scheduler.
Manual trigger	Run on demand by staff or by the API.

Actions

ACTION	EFFECT
Send an approved message	Sends a templated message through the business channel, subject to rate limits.
Assign a conversation	Sets the assignee on a thread.
Create a task	Adds a task with title, description, priority and assignee.
Schedule a follow-up	Queues a message for a future time, sent by the platform.
Prepare a quotation	Creates a quotation from an order.
Notify staff	Writes a notification visible in the console.
Update an authorised field	Changes a whitelisted field on an allowed record type.
Request human approval	Queues an approval; downstream actions wait for a decision.
Initiate an approved business tool	Runs a named tool with fixed arguments — never arbitrary code.
Tag the customer	Adds a tag for segmentation and later conditions.

Conditions and limits

Conditions are evaluated before any action and can reference the triggering record (order total, payment status, customer tags, stock level, message text). A run executes at most ten actions — a loop guard that stops a misconfigured rule from becoming a message storm. **Request human approval** pauses a run until someone decides; the approval queue is visible on **Overview** and through `/api/approvals`.

Creating and watching rules

Create a workflow asks for a name, a trigger, conditions and actions, plus a switch for whether a human must approve the whole run. The **Execution log** below lists recent runs with trigger, result, duration and error text. Failed runs are counted on the Overview so a silent breakage does not go unnoticed.

> NO CODE, NO EXCEPTIONS
>
> A workflow cannot execute JavaScript, shell commands or SQL. The “initiate an approved business tool” action is limited to the same named tools the assistant uses, with arguments validated by the same schemas. That is what makes it safe to let a manager build automation.

SECTION 17

Business Intelligence

Ten report builders over your own data, from the accounting summary to a demand forecast that states its own uncertainty.

What the page presents

REPORT	QUESTION IT ANSWERS
Accounting summary	Revenue, collected, outstanding, expenses and net position for a date range.
Revenue by product	Which items actually make money, ranked.
Lead funnel	New → contacted → booked → closed, with drop-off at each step.
Forecast	Expected demand per item for the next days, from recent history.
Inventory risk	Items likely to run out before the next restock.
AI usage	Runs, tokens and estimated cost by provider and model.
Revenue trend	Daily paid revenue and order counts.
Orders needing attention	Stuck, unpaid, disputed or overdue orders.
Conversations needing attention	Unanswered or escalated threads.
Overview metrics	The twelve counters on the dashboard, computed in one pass.

Reading the forecast honestly

The forecast is a moving average over recent paid orders, and it refuses to produce a number until there are at least fourteen days of history. When it does produce one it is a planning hint, not a promise: seasonality, festivals and promotions are not modelled. The page says so, and so does this document.

Exports

Leads, customers and orders export to CSV from their own pages. Reports are displayed in the console and available as JSON from `/api/reports/…` for a spreadsheet, a BI tool or a scheduled job.

> ONE DEFINITION OF REVENUE
>
> Every report uses the same definition: the sum of verified payments, excluding refunds. That is why the dashboard, the accounting summary and the CSV export reconcile without manual adjustment.

SECTION 18

Knowledge Base

The facts the assistant is allowed to state: policies, delivery terms, service areas, opening hours, return rules — searchable, chunked and cited.

Documents

Each document has a title, a category, a body and a status. Saving splits the body into overlapping chunks, indexes them for full-text search, and records who changed what. Re-indexing is available per document and for the whole business after bulk edits.

FIELD	PURPOSE
Title	Human label, also used when the assistant cites the source.
Category	Grouping such as *delivery*, *returns*, *warranty*.
Content	Plain text. Short, factual paragraphs retrieve better than one long essay.
Status	Draft documents are not used in replies until published.

How it is used

On each turn the engine runs a search over the customer's message (with language-aware normalisation), takes the best few chunks and includes them in the prompt as reference material. If nothing relevant is found, the assistant is instructed to say it does not know rather than invent. A legacy key/value knowledge base from ChatCloser 1.x can be imported from **Settings** and is kept as a fallback for short facts.

> WRITE FOR RETRIEVAL
>
> One fact per paragraph, with the words a customer would use. “Delivery inside Kathmandu is free above NPR 2,000 and takes 2–3 working days” retrieves far better than a page of brand story. If the assistant keeps answering a question wrongly, the fix is almost always a shorter, more explicit knowledge document.

SECTION 19

Tasks, reminders and notifications

The work that is not a conversation: things someone must do, messages that must be sent later, and the alerts that keep a team honest.

Tasks

Tasks carry a title, description, priority, assignee, due time and one of five states: `open`, `in_progress`, `waiting`, `completed`, `cancelled`. The assistant can create one through `schedule_follow_up`; workflows create them with the **Create a task** action; staff use `/api/tasks`. The open count is on the Overview.

Scheduled messages

A scheduler stores messages with a send time and status. It sends only what is due, marks each attempt, and records the error if a send fails. This is how follow-ups happen without anyone remembering: the assistant or a workflow schedules, the platform sends.

Notifications and approvals

ITEM	BEHAVIOUR
Notifications	Per-user or business-wide alerts with category and severity; unread count is tracked and each can be marked read.
Approvals	A queue of decisions requested by a workflow or a tool marked `approvalRequired`. Approving lets the action proceed; rejecting closes it without side effects.
Escalations	Created when a keyword matches or the assistant calls `request_human_takeover`; the thread switches to human mode and the customer sees the escalation message.

> NO SILENT FAILURES
>
> A failed send, a rejected approval and a failed workflow all produce a visible record. The design rule is that background work must either succeed or leave a trace someone can find on the Overview or in the audit log.

SECTION 20

Team and permissions

Five built-in roles, per-permission overrides, and a server that checks every request rather than trusting a hidden link.

Adding a member

Add a team member takes a username (three characters or more), a display name, a role and a password of at least eight characters. The password is hashed with scrypt and never stored or shown again. Removing a member keeps their audit history.

Permission matrix

● granted by the role · ○ not granted · Owner is always all permissions.

PERMISSION	OWNER	MANAGER	STAFF	ACCOUNTANT	VIEWER
Dashboard view	●	●	●	●	●
Leads view / edit	●	●	●	○	○
Conversations view	●	●	●	○	●
Conversations reply	●	●	●	○	○
Conversations takeover	●	●	○	○	○
Customers view	●	●	●	●	●
Customers edit	●	●	○	○	○
Customers export	●	○	○	●	○
Customers delete	●	○	○	○	○
Catalogue view	●	●	●	○	●
Catalogue edit	●	○	○	○	○
Inventory view	●	●	●	○	○
Inventory edit	●	○	○	○	○
Orders view	●	●	●	●	●

PERMISSION	OWNER	MANAGER	STAFF	ACCOUNTANT	VIEWER
Orders create / edit / cancel	●	●	○	○	○
Bookings view	●	●	●	○	●
Bookings edit	●	●	○	○	○
Quotes view	●	●	●	●	●
Quotes edit	●	●	○	○	○
Invoices view	●	●	●	●	●
Invoices edit	●	○	○	●	○
Payments view	●	●	○	●	●
Payments manage / refund	●	○	○	●	○
Reports view	●	●	○	●	●
AI view	●	●	○	○	○
AI edit	●	○	○	○	○
Knowledge view	●	●	●	○	○
Knowledge edit	●	○	○	○	○
Workflows view	●	●	○	○	○
Workflows edit	●	○	○	○	○
Integrations view	●	●	○	○	○
Integrations edit	●	○	○	○	○
Team view	●	●	○	○	○
Team edit	●	○	○	○	○
Settings view	●	●	○	○	○
Settings edit	●	○	○	○	○
Audit view	●	○	○	○	○
Backups view / manage	●	○	○	○	○

Overrides

A user record can carry a permission map on top of the role: `{"payments.view": true}` grants one extra area to a staff member, `{"catalog.edit": false}` removes one. Overrides are checked after the role, so the result is predictable. Owners cannot be reduced below full access.

> HIDING IS NOT SECURITY
>
> Navigation items are hidden for convenience. The enforcement point is the server: every console route declares the permission it needs, every API endpoint checks it, and every assistant tool declares the permissions the business must have granted. A crafted request gains nothing.

SECTION 21

Security and audit logs

A chronological record of privileged actions: who, what, when, and which record. Written by the same code that performs the action, not by the UI.

What is recorded

Sign-ins and failed sign-ins, business and module changes, customer and catalogue edits, order state changes, payment verification and approval, quotation and invoice creation, expense entry, workflow changes and runs, team changes, backup creation and restore, AI configuration changes, and assistant tool calls that change data.

Anatomy of an entry

FIELD	CONTENT
When	UTC timestamp, rendered in the workspace timezone.
Actor	Username, or a system identity such as `customer`, `ai`, `workflow` or `system`.
Action	Dotted identifier, for example `order.state_changed` or `payment.verified`.
Details	Structured JSON: record ids, previous and new values where relevant, totals.
Business	Tenant id, so a shared database still separates workspaces.

Using the log

- Reconstruct what happened to one order when a customer disputes it.
- Confirm that a payment was verified by the provider rather than approved by hand.
- See which staff member changed a price, and when, before a complaint.
- Check that the assistant only used tools you enabled.

> RETENTION
>
> Audit entries are kept indefinitely by default. They are small, and the record is often the point. If regulation requires pruning, do it deliberately with a SQL statement against a backup, not by truncating the table.

SECTION 22

Settings and backups

The company profile that shapes every reply and document, the modules that decide which pages exist, and encrypted backups you can actually restore.

Company profile

SETTING	EFFECT
Business name	Used in replies, documents and the console header.
Locale	Language and number formatting defaults (`en`, `ne`).

SETTING	EFFECT
Currency	Display currency for prices and totals. Existing records keep their stored currency.
Country / timezone	Phone validation and timestamp rendering; default `NP` / `Asia/Kathmandu`.
Industry template	Enables the template's modules and intake fields. Applying one never deletes existing records.
Tax enabled / rate / inclusive	Whether prices include tax, and the rate applied when they do not.
Delivery fee / free above	The delivery charge and the threshold at which it disappears; both are quoted by `get_delivery_options`.
Require customer confirmation	When yes, staff also cannot confirm a draft without asserting the customer approved. The assistant always needs the customer's own words.
Escalation keywords / message	Words that hand a thread to a human immediately, and what the customer is told.
Away message	Sent outside business hours when a human is needed.
Assistant instructions	Extra system text: tone, promises, phrases to avoid. See §27.

Modules

Each module is a switch: invoices, quotations, inventory, bookings, workflows, payments and so on. Disabling one hides its navigation item and stops the assistant using its tools. Data is preserved — turning a module back on restores the pages and the history.

Backups

Create backup takes a consistent snapshot with `VACUUM INTO` — no downtime, no half-written file — then encrypts it with AES-256-GCM under the workspace encryption key and stores it in the data folder's `backups` directory. The list shows name, size, time and whether it is encrypted.

ACTION	BEHAVIOUR
Create backup	Snapshot plus encryption, audited with size and name.
Restore	Verifies the GCM tag, then replaces the live database. Stop writing before restoring; the app reopens the restored file.
Copy off the machine	Backups are only safe once they exist somewhere else. Copy the `backups` folder to storage you control.

> **THE KEY IS PART OF THE BACKUP**
>
> Encrypted backups cannot be opened without the encryption key from `secrets.json` (desktop) or `ENCRYPTION_KEY` (server). Store the key in a password manager, separately from the database. Losing it makes every encrypted backup permanently unreadable.

SECTION 23

Account

Your own record: display name, username and password. No business data, no other people's credentials.

What you can change

FIELD	RULE
Display name	Shown in the sidebar, on notes and in audit entries.

FIELD	RULE
Password	Requires the current password, and at least eight characters. Stored as a scrypt hash.
Username	Set by an owner on **Team**; not editable from this page.

Sessions

Sessions last twelve hours and live in the database, so restarting the app does not sign you out. **Sign out** destroys the session server-side. Changing a password does not retroactively end other sessions; if you suspect a shared machine, sign out there explicitly.

> FIRST THING AFTER INSTALLATION
>
> Change the generated owner password, then delete `first-run-credentials.txt`. That file is the only place the original password is written in clear text.

PART III

The assistant

How a reply is produced, which model providers are supported, the seventeen tools the assistant may call, and how to teach it your business without teaching it to improvise.

1. How the assistant works
2. Providers, models and cost control
3. Tool reference
4. Teaching it your business

SECTION 24

How the assistant works

A deterministic pipeline around a probabilistic model: context is assembled by code, actions are validated by code, and the model only chooses words and tool requests.

The turn, step by step

1. **Language.** The incoming text is normalised and classified — English, Nepali script, or romanised Nepali — and an intent is detected (greeting, price question, order status, payment, complaint, handover request).
2. **Context.** The business profile, enabled modules, current date and timezone, delivery rules, catalogue excerpts for anything that looks like an item, knowledge search results and the last twelve messages are assembled.
3. **Instruction.** The system prompt states the role, the hard rules (never invent a price, never claim a payment, always confirm before ordering), the escalation policy and any extra instructions you wrote.
4. **Model call.** The provider adapter sends a chat completion with the tool schemas attached. Temperature and token limits come from **AI Configuration**.
5. **Tool loop.** If the model requests tools, each call is schema-validated, permission-checked, executed and returned as structured data. The model may call several in sequence; the loop is bounded.
6. **Reply check.** Empty output, leaked internal markers and over-long replies are rejected and retried or replaced with a safe fallback.
7. **Delivery.** The text is stored, then sent with a typing indicator, a human-like pause and rate limiting.

What the model is told it may not do

- Quote a price that did not come from a tool call in this conversation.
- Create an order without the customer's explicit confirmation of the exact summary.
- Say a payment is complete based on the customer's word, a screenshot or its own reasoning.
- Promise a delivery date before checking stock.
- Discuss another customer, another order, or internal notes.
- Improvise a policy that is not in the knowledge base — it must say it does not know and offer a human.

Escalation

Three independent paths hand a conversation to a person:

PATH	TRIGGER
Keyword	The message contains one of the configured escalation keywords, matched before the model is even called.
Model decision	The model calls `request_human_takeover` — for a complaint, a refund, a safety issue, or when it is not confident.
Staff action	Someone presses **Take over** in the inbox.

All three set the thread to human mode, send the escalation message, and create a high-priority task so the handover is not forgotten. The assistant stays silent until someone presses **Resume AI**.

Failure behaviour

If the provider is unreachable, the key is invalid, or the model returns nothing usable, the customer receives a short apology and the thread is flagged for a human. The error is logged with provider and model, never with the API key. The assistant does not retry indefinitely, because a customer waiting in silence is worse than a fast handover.

> HISTORY WINDOW
>
> Only the last twelve messages are sent with each turn. That keeps cost predictable and stops a long thread from drifting, but it also means a fact stated fifty messages ago is not in context — put durable facts in the knowledge base instead of relying on memory.

SECTION 25

Providers, models and cost control

Eight adapters behind one interface. Bring your own key, or run a local model and keep every word on your own machine.

Supported providers

PROVIDER	KEY TYPE	NOTES
Anthropic (Claude)	API key	Strong instruction-following and tool use; good default for sales conversations.
OpenAI (GPT)	API key	Broad model range; reliable JSON/tool behaviour.
Google (Gemini)	API key	Has a free tier; useful for a low-volume start.
DeepSeek	API key	Low cost per token; good for high-volume, simple catalogues.
OpenRouter	API key	One key for many models; switch models without changing providers.
NVIDIA NIM	API key	Has a free tier; hosted open models.
Groq	API key	Free tier without a card; very low latency.
Local model (Ollama)	None	Data never leaves the computer. Needs a capable machine; slower and less fluent than hosted models.

Configuration fields

FIELD	MEANING
Provider	Which adapter to use. Switching providers keeps the old key encrypted but unused.
Model	Exact model identifier, for example `claude-sonnet-4-5` or `llama3.1:8b`.
API key	Encrypted with AES-256-GCM before it touches the database; never shown again after saving.
Temperature	0 to 2, default 0.4. Lower is more literal; higher is chattier and riskier for prices.
Max tokens	64 to 8,000, default 500. Replies are short by nature; 300–600 is usually right.
Operator role	The persona paragraph: who the assistant is, how formal, what language to prefer.
Local only	Refuses to call a hosted provider even if one is configured — a hard privacy switch.
Local base URL	For Ollama on another machine or a non-default port.

Test, then save

Test sends a minimal prompt through the selected adapter and reports the exact error if it fails — invalid key, wrong model name, quota exhausted, host unreachable. Nothing is saved until the test passes and you press **Save**. For local models, the test also reports whether the configured model is actually pulled.

Cost control

- Start with a cheap or free-tier model; move up only if reply quality is the bottleneck.
- Keep **Max tokens** near 500. Long replies cost more and read worse on a phone.
- Watch **AI runs (30 days)** on the Overview and the usage block in **AI Command Center**.
- Disable tools you do not use; fewer schemas means fewer tokens on every turn.
- Use the knowledge base instead of a giant instruction paragraph — retrieved chunks are cheaper and more accurate than a permanent wall of text.

> KEYS ARE SECRETS
>
> Anyone who can read the database file and the encryption key can decrypt stored provider credentials. Protect the data folder with OS permissions, keep backups encrypted, and rotate a key immediately if a laptop is lost. See §30.

SECTION 26

Tool reference

Seventeen named operations. Each declares its schema, the permission it needs, whether it writes, and whether a human must approve it.

TOOL	EFFECT	PERMISSION	WHAT IT DOES
`search_products`	read	`catalog.view`	Finds items by name, description, SKU or barcode; returns verified names and prices.
`get_product_details`	read	`catalog.view`	Full detail for one item: price, tax, unit, stock, attributes.
`check_stock`	read	`inventory.view`	Available quantity now, and whether a requested quantity can be supplied.
`calculate_quote`	read	`catalog.view`	Prices a list of lines with the backend's discount, tax and delivery rules.
`get_delivery_options`	read	`orders.view`	Delivery charge, free-delivery threshold and service area.
`create_order_draft`	write	`orders.create`	Prepares a priced summary. No order, no charge, no stock movement.
`submit_order_for_confirmation`	write	`orders.create`	Sends the summary and asks the customer to confirm it.
`confirm_customer_order`	transactional	`orders.create`	Creates the real order — only with the customer's own confirmation text. Re-prices every line first.
`get_order_status`	read	`orders.view`	State and payment status of an existing order.
`cancel_order`	transactional	`orders.cancel`	Cancels before dispatch and releases reserved stock. Approval required.
`create_booking`	write	`bookings.edit`	Books an appointment, visit or table with the details the business requires.

TOOL	EFFECT	PERMISSION	WHAT IT DOES
`create_support_ticket`	write	`orders.create`	Raises a ticket for a complaint, fault or refund request the assistant cannot resolve.
`schedule_follow_up`	write	`orders.create`	Queues a later follow-up; the platform sends it, not the model.
`get_payment_status`	read	`payments.view`	The provider-verified payment state — never what the customer claims.
`request_human_takeover`	write	`conversations.takeover`	Stops automatic replies and hands the thread to staff with a summary.
`capture_lead`	write	`leads.edit`	Records a qualified lead once a need plus a name or number is known.
`close_sale`	transactional	`orders.create`	Legacy 1.x compatibility: records a payment link for one priced item. New conversations use drafts.

Schema validation

Every tool has an input schema and an output schema. Arguments are checked for type, length, enum membership and required fields before execution; unknown properties are rejected rather than ignored. The model sees a structured result — `{ ok, ... }` or a typed error — and must reason from it, which is why a failed confirmation produces "please confirm the new total" rather than a hallucinated success.

> PER-BUSINESS SWITCHES
>
> Tools can be disabled individually on **AI Configuration**. Disabling `cancel_order`, for example, means the assistant must escalate cancellations to a person, while everything else keeps working. The tool list on **AI Command Center** shows what is available.

SECTION 27

Teaching it your business

A ten-step go-live checklist, in the order that produces a working assistant fastest.

Go-live checklist

1. **Company profile.** Name, currency, timezone, tax, delivery fee and free-delivery threshold. These numbers are quoted to customers, so they must be right.
2. **Industry template.** Pick the closest match; it enables the right modules and adds intake fields for orders and bookings.
3. **Catalogue.** Add every sellable item with the exact name, price and unit. If you sell variants, add them — a missing variant becomes "I don't have that".
4. **Knowledge base.** One document per topic: delivery, returns, warranty, payment methods, opening hours, service area, lead times. Short factual paragraphs.
5. **Escalation policy.** Set keywords (for example *refund, complaint, manager, lawyer, damaged*) and the message the customer sees when a human takes over.
6. **Away message.** What a customer hears outside business hours, so silence is never the answer.
7. **Operator role and extra instructions.** Tone, language preference, things never to promise. Keep it under a few paragraphs.
8. **AI key and test.** Provider, model, key, **Test**, **Save**. Start with temperature 0.4 and 500 tokens.
9. **WhatsApp.** Scan the QR, send a test message from another phone, and watch the thread in the inbox.

10 **Team and backups.** Create staff accounts with the smallest role that works; take a backup before inviting real customers.

Writing instructions that work

INSTEAD OF	WRITE
“Be helpful and friendly.”	“Reply in the customer's language. Keep answers under 60 words. Use the customer's name once, at most.”
“Offer good prices.”	“Never offer a discount unless the catalogue has a discounted price. If asked, say a manager can discuss bulk pricing.”
“Handle complaints.”	“For any complaint, damaged item or refund request, apologise once and call request_human_takeover immediately.”
“Know our delivery.”	Add a knowledge document: “Delivery inside Kathmandu: NPR 100, free above NPR 2,000, 2–3 working days. Outside the valley: 4–6 days, quoted per order.”

Testing before real customers

- Message the linked number from a personal phone and ask for an item you know the price of. The quoted figure must match the catalogue exactly.
- Ask for something not in the catalogue. It should say so and offer a human, not invent an item.
- Ask “where is my order?” with no order. It should ask for the order number, not guess.
- Confirm an order, then check **Orders**: a draft becomes an order only after your yes, and stock is reserved.
- Send a keyword such as “refund”. The thread should switch to human mode and the customer should see the escalation message.
- Open **Security & Audit Logs** and confirm each of those actions is recorded.

> THE SINGLE BIGGEST QUALITY LEVER
>
> A complete, well-named catalogue. Most “the assistant is dumb” reports are really “the item is spelled differently in the catalogue than the customer spells it”. Add synonyms and common misspellings to the item description; retrieval matches on description text.

PART IV

Channels and money

The two connections that touch the outside world: the WhatsApp session that carries conversations, and the payment providers that turn a confirmed order into collected money.

1. WhatsApp
2. Payment providers in depth

SECTION 28

WhatsApp

Linking, status, reconnection, pacing — and the honest risks of running on the WhatsApp Web protocol.

Linking a number

1. Open **Integrations** → **WhatsApp**. The status is *waiting* and a QR code is shown.
2. On the business phone: WhatsApp → **Settings** → **Linked devices** → **Link a device**.
3. Scan the code. The page refreshes the code every five seconds, so an expired code replaces itself; you do not need to reload.
4. Within a few seconds the status becomes *connected*, the QR disappears and the hint confirms the assistant is answering.

Status values

STATUS	MEANING
starting	Process booting, socket not yet created.
waiting_for_qr	No session on disk; a QR is available.
connected	Session valid; messages flow in both directions.
disconnected	Socket closed; the client reconnects with backoff. If the session was revoked, a new QR appears.
disabled	WhatsApp switched off deliberately with `WHATSAPP_ENABLED=false`.

What happens to messages

MESSAGE TYPE	BEHAVIOUR
Text	Processed, stored, answered by the assistant or routed to a human.
Your own outgoing	Ignored by the assistant (no reply loops); still visible in WhatsApp itself.
Status broadcasts	Ignored.
Images, voice notes, documents	Not processed in this release. The message is not answered; a human should follow up. See §36.

Pacing and safety

Sends are paced to look human: a typing indicator, a 1.2–3 second pause, and two rate limits — six messages per chat per minute and thirty globally per minute. When a limit is hit, the reply waits rather than failing. Follow-ups that go unanswered are nudged after 24 hours by default (`NUDGE_AFTER_HOURS`).

Reconnecting and moving numbers

- Restarting the app does not require a new scan: the session is stored in `auth_info` in the data folder.
- **Log out WhatsApp** clears that session and asks you to link again — use it when the phone is replaced or the number changes.
- Linking the same number on a second ChatCloser installation replaces the session on the first. One number, one installation.
- If the phone is offline for a long period, WhatsApp may require a fresh link.

READ THIS BEFORE A BULK CAMPAIGN

This is not the official WhatsApp Business API and it has no template approval, no green tick and no guaranteed deliverability. Bulk or unsolicited messaging is the fastest way to get a number restricted. Use a dedicated business number, message only people who wrote to you first, keep volumes conversational, and never let a workflow send to a purchased list. Group chats are not a supported surface — keep the business number out of groups.

SECTION 29

Payment providers in depth

Five ways to collect money, one rule: a payment is paid only when the provider or an authorised human confirms it.

Adapters

PROVIDER	CREDENTIALS	FLOW
Stripe	Secret key, webhook signing secret	Hosted Checkout session with success and cancel URLs. Webhook events are signature-verified with the raw request body.
eSewa	Merchant code, secret key	Signed payment form or redirect, then server-side verification of the callback. Sandbox and production hosts are built in.
Khalti	Merchant key (public), secret key	Initiate an e-payment, redirect to the returned payment URL, then look up the transaction by `pidx`.
Cash on delivery	None	No online charge. A staff member approves the collected cash as a manual payment.
Bank transfer	None (instructions text)	Customer transfers, sends a receipt, staff approve the payment with a note.

Choosing the default

Integrations → Payment providers stores the credentials (encrypted) and selects which provider is used for new payment links: `stripe`, `esewa`, `khalti`, `cash_on_delivery` or `bank_transfer`. The API can override the provider and pass an `environment` of `sandbox` or `production` when creating or verifying a payment; sandbox is the default so a test never moves real money.

Verification, not trust

1. **Create.** A payment row is written with provider, amount, currency and a reference, tied to the order.
2. **Pay.** The customer completes the provider's flow and returns to the success URL.
3. **Verify.** The server calls the provider's lookup API (or verifies the webhook signature) and compares amount, currency and reference.
4. **Apply.** Only a verified success writes `succeeded`, marks the order paid, fires the payment-verified trigger and sends the receipt message.
5. **Record the event.** The provider event id is stored so a retried webhook cannot double-apply.

Webhooks

Provider callbacks arrive on the control port: `POST /webhooks/stripe` (raw JSON, signature header) and `POST /webhooks/nepal/:provider` (form-encoded). Because the control API listens on loopback by default, a provider on the internet

cannot reach it directly. For automatic confirmation you need a public HTTPS endpoint — a reverse proxy that forwards only those paths, or a tunnel — while keeping the rest of the control API private. Without a webhook, use **Verify** on the Payments page, which performs the same lookup on demand.

Refunds and manual receipts

- A refund is recorded against the original payment and reconciled; the order moves to `refunded` through the allowed transition.
- Manual receipts are never auto-approved. Staff approve with a note, and the approver is recorded in the audit log.
- Amount mismatches — a customer pays a different figure — are left in `requires_review` rather than guessed at.

> SANDBOX FIRST
>
> Every provider has a test mode. Run one complete order through sandbox before switching the default to production, and check that the order becomes paid, the invoice is generated and the payment-verified workflow fires. A production key in a sandbox flow fails in confusing ways; the reverse moves real money.

PART V

Administration

The part that keeps it running: the security model, where the data lives, every configuration variable, deployment behind TLS, the desktop shell, and what to do when something looks wrong.

1. Security model
2. Data, backup and migration
3. Configuration reference
4. Deployment
5. The desktop application
6. Troubleshooting
7. Limitations and non-goals
8. Upgrading from ChatCloser 1.x
9. Glossary
10. Support, licence and credits

SECTION 30

Security model

What is protected, how, and what remains your responsibility. Written for the person who has to answer a customer's security questionnaire.

Authentication and sessions

CONTROL	IMPLEMENTATION
Password storage	scrypt with a per-user salt; plaintext is never stored or logged. Minimum eight characters.
Session cookie	`chatcloser.sid`, HttpOnly, SameSite=Lax, twelve-hour lifetime, `Secure` when `COOKIE_SECURE=true`.
Session store	SQLite, so sessions survive restarts and are not held in process memory.
Session fixation	The session id and CSRF token are regenerated on successful sign-in.
Sign-out	Destroys the server-side session, not just the cookie.

Request protection

- **CSRF.** A per-session token is injected into every form as `_csrf` and required on every non-safe method; the JSON API sends it as `X-CSRF-Token`.
- **CSP.** `default-src 'self'` with nonce-based scripts, no inline event handlers, `object-src 'none'`, `frame-ancestors 'none'`.
- **Headers.** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: same-origin`, restrictive `Permissions-Policy`.
- **Input.** Every value passes a typed validator; SQL is always parameterised; all customer text is HTML-escaped on output.
- **Body limits.** 1 MB for form and JSON bodies, so a huge payload cannot exhaust memory.

Network exposure

LISTENER	DEFAULT	CHANGE IT WHEN
Console (3001)	`127.0.0.1`	Staff must reach it from other machines — put TLS in front and set `DASHBOARD_HOST`.
Control API (3002)	`127.0.0.1`	Almost never. Expose only the webhook paths, through a proxy that filters them.

Secrets at rest

Provider keys and payment credentials are encrypted with AES-256-GCM before storage. The key comes from `secrets.json` (desktop) or `ENCRYPTION_KEY` (server) and is never written into the database. Backups are encrypted with the same key. In server mode, startup fails if the key or session secret is missing or a known placeholder — a deliberate refusal to run insecurely.

Threat model

SCENARIO	MITIGATION
Stolen laptop	OS disk encryption plus the data-folder key; rotate provider keys and sign out WhatsApp if the device is gone.
Malicious customer message	Prompt injection cannot execute code; tools are schema-bound and permission-checked. A customer cannot read another customer's data.

SCENARIO	MITIGATION
Rogue staff member	Least-privilege roles, per-permission overrides, full audit trail, destructive actions behind explicit forms.
Forged payment webhook	Signature verification (Stripe) or server-side lookup (eSewa, Khalti); event ids prevent replays.
Leaked backup	AES-256-GCM encryption; without the key the file is opaque.
Phishing of an operator	No email-based password reset to phish; passwords change only from inside a signed-in session.

Hardening checklist

- [ ] Change the owner password; delete `first-run-credentials.txt`.
- [ ] Give each person their own account; no shared logins.
- [ ] Use the smallest role that lets someone do their job.
- [ ] Enable disk encryption on the machine holding the data folder.
- [ ] Put HTTPS in front before exposing the console to a network; set `COOKIE_SECURE=true`.
- [ ] Keep `ENCRYPTION_KEY` in a password manager, not in the repository.
- [ ] Take backups and copy them off the machine.
- [ ] Review the audit log monthly for unexpected approvals or role changes.
- [ ] Rotate AI and payment keys after any staff departure with access.

SECTION 31

Data, backup and migration

One SQLite file, a handful of sibling folders, and a restore path that has been tested. Moving a workspace between machines is a folder copy.

Where the data lives

LOCATION	CONTENTS
Desktop: `%APPDATA%\ChatCloser`	The complete workspace. Portable and installed editions share it.
Server: working directory	Default data folder; override with `CHATCLOSER_DATA_DIR`.
`data.sqlite`	All business data, sessions and settings. WAL mode adds `-wal` and `-shm` while running.
`secrets.json`	Encryption key and first-run secrets (desktop).
`auth_info/`	WhatsApp session credentials. Losing it means scanning the QR again.
`backups/`	Encrypted snapshots created from the console.
`logs/`	Application logs.
`desktop.log`	Desktop shell log, including startup failures before the console exists.

Schema overview

Six migrations create the core tables: businesses and modules, users and sessions, customers and tags, conversations and messages, leads and tasks, catalogue and inventory, orders and order items, payments and provider events, quotations, invoices and expenses, workflows and runs, approvals, notifications, knowledge documents and chunks with a full-text index, scheduled messages, audit log, and backups metadata. Foreign keys are enforced; every business-owned table carries `business_id` and every query filters on it.

Backup and restore

1. **Create backup** in **Settings & Backups**. It snapshots with `VACUUM INTO`, so the file is consistent without stopping the app, then encrypts it.
2. **Copy it off the machine** — to a NAS, object storage or another disk. A backup on the same disk survives a mistake, not a failure.
3. **Restore** from the same page. The GCM tag is verified before the live database is replaced; a corrupted or wrong-key file is refused.
4. **Verify** after restoring: sign in, open an order, confirm a payment's verified status and check that WhatsApp reconnects.

Moving to another computer

1. Quit ChatCloser on both machines (tray → **Quit**).
2. Copy the entire data folder to the same path on the new machine.
3. Start the app there. Database, keys, settings and the WhatsApp session come with it — no QR scan.

If the new machine uses a different Windows account, copy the folder into that account's `%APPDATA%` with an administrator account and then start the app as the intended user.

> NEVER COPY A LIVE DATABASE FILE
>
> Copying `data.sqlite` while the app is running can capture a half-written page, and the `-wal` file holds committed data that the main file does not yet contain. Either quit first or use **Create backup**, which is the supported path.

SECTION 32

Configuration reference

Every environment variable the application reads, with its default. Desktop installs need none of them; server installs need the two secrets.

Paths and identity

VARIABLE	DEFAULT	PURPOSE
`CHATCLOSER_DATA_DIR`	cwd / userData	Folder for the database, auth session, backups and logs.
`CHATCLOSER_DB_PATH`	dataDir/data.sqlite	Explicit database file path; overrides the folder default.
`BUSINESS_ID`	1	Tenant id used by a single-business install.
`BUSINESS_NAME`	Your Company Name	Initial business name at provisioning.
`INDUSTRY_TEMPLATE`	retail	Template applied on first run.
`ADMIN_USERNAME`	admin	Owner username created on first run.
`ADMIN_PASSWORD`	generated	Owner password; leave empty to generate and print once.

Secrets and session

VARIABLE	DEFAULT	PURPOSE
ENCRYPTION_KEY	required	32-byte key (base64) for provider credentials and backups. Startup fails without it in server mode.
SESSION_SECRET	required	Signs session cookies. Changing it signs everyone out.
COOKIE_SECURE	false	Marks cookies Secure. Enable only when served over HTTPS.
CHATCLOSER_STRICT_SECRETS	true (server)	Refuse to start with missing or placeholder secrets.
CHATCLOSER_ALLOW_INSECURE_SECRETS	false	Explicit escape hatch for a throwaway local test.

Network

VARIABLE	DEFAULT	PURPOSE
DASHBOARD_PORT	3001	Console port.
DASHBOARD_HOST	127.0.0.1	Console bind address. `0.0.0.0` exposes it to the network — use TLS.
BOT_CONTROL_PORT	3002	Control API and payment webhook port.
BOT_CONTROL_HOST	127.0.0.1	Control API bind address.
CHATCLOSER_SERVER_MODE	false	Marks a headless always-on deployment.

Messaging and payments

VARIABLE	DEFAULT	PURPOSE
WHATSAPP_ENABLED	true	Set false to run the console without the WhatsApp client.
RATE_LIMIT_PER_CHAT_PER_MIN	6	Messages per conversation per minute.
RATE_LIMIT_GLOBAL_PER_MIN	30	Messages across all conversations per minute.
NUDGE_AFTER_HOURS	24	Hours before an unanswered follow-up is nudged.
PAYMENT_SUCCESS_URL	—	Return URL after a successful provider payment.
PAYMENT_CANCEL_URL	—	Return URL when the customer cancels.
NODE_ENV	development	`production` turns on stricter secret handling.

> **WHERE SETTINGS REALLY LIVE**
>
> Business behaviour — name, currency, tax, delivery, hours, assistant persona, provider keys — lives in the database and is edited in the console. Environment variables only decide how the process starts and where it listens. Changing a business setting does not require a restart; changing an environment variable does.

SECTION 33

Deployment

An always-on server behind TLS, with a service unit, a reverse proxy, backups and a health check.

Service unit

```
[Unit]
Description=ChatCloser
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=chatcloser
WorkingDirectory=/opt/chatcloser
EnvironmentFile=/opt/chatcloser/.env
Environment=NODE_ENV=production
ExecStart=/usr/bin/node scripts/server.js
Restart=always
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Reverse proxy

```
server {
    listen 443 ssl http2;
    server_name chat.example.com;

    ssl_certificate     /etc/letsencrypt/live/chat.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/chat.example.com/privkey.pem;

    client_max_body_size 2m;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # payment callbacks only — nothing else from the control API
    location /webhooks/ {
        proxy_pass http://127.0.0.1:3002;
        proxy_set_header Host $host;
    }
}
```

Set `COOKIE_SECURE=true` once HTTPS works, and `DASHBOARD_HOST=127.0.0.1` so the proxy is the only way in. Forwarding `X-Forwarded-Proto` lets Express mark cookies correctly behind TLS.

Health and logs

CHECK	EXPECTED
`GET /health`	`{"status":"ok","whatsapp":"connected","uptimeSeconds":…}` on the console port.
`GET :3002/status`	WhatsApp connection state only — useful for a local monitor.
`logs/`	Application log; desktop installs also write `desktop.log`.
`Overview`	Failed workflows, unverified payments and unanswered threads are the operational health signals.

Backups on a schedule

The console creates encrypted backups on demand. For unattended backups, call the API from cron with a signed-in session, or snapshot the data folder at the filesystem level while the app is stopped. Always copy backups off the host.

Upgrading

```
cd /opt/chatcloser
npm ci
npm run migrate
systemctl restart chatcloser
curl -s http://127.0.0.1:3001/health
```

Take a backup before upgrading. Migrations are additive and run automatically at startup; the explicit `npm run migrate` surfaces a failure before the service restarts.

SECTION 34

The desktop application

A thin Electron shell around the same backend: a window, a tray icon, a data folder and a first-run dialog. No separate feature set.

Behaviour

BEHAVIOUR	DETAIL
Startup	Starts the backend in-process, waits for the console to answer, then shows the window. WhatsApp starts on its own and reaches *waiting* or *connected*.
Closing the window	Hides to the tray; the assistant keeps answering.
Tray menu	Open console, open data folder, launch on startup, quit.
Quit	Stops the backend and the WhatsApp socket cleanly.
Second instance	Focuses the existing window instead of starting a second backend on the same ports.
Single business	One workspace per installation; the desktop shell has no tenant switcher.

Build outputs

FILE	PURPOSE
`ChatCloser-Setup-2.0.0-x64.exe`	Per-user installer, desktop and Start Menu shortcuts, uninstaller.

FILE	PURPOSE
`ChatCloser-Portable-2.0.0-x64.exe`	Single file, unpacks to temp, shares the same data folder.
`win-unpacked/`	Unpacked build for inspection and smoke testing.

Smoke testing a build

The shell supports an automated smoke mode used before shipping a build: it signs in, loads a list of pages, writes a result file and exits. Set `CHATCLOSER_DESKTOP_SMOKE` to `1` or `2` with `CHATCLOSER_SMOKE_OUT`, `CHATCLOSER_SMOKE_USER`, `CHATCLOSER_SMOKE_PASSWORD` and optionally `CHATCLOSER_SMOKE_PAGES`. A passing run reports every page with a 200 status and confirms `preload:true` with `nodeInRenderer:false`.

Signing and updates

The current build is unsigned, so Windows shows SmartScreen on first run. Production distribution should code-sign the installer; the application has no auto-updater, so updates are a new installer run over the old one, with data preserved. Uninstalling removes the program and shortcuts but leaves `%APPDATA%\ChatCloser` untouched.

> TRAY OR TASK MANAGER
>
> If the window is gone but the assistant still replies, it is in the tray. If a start seems to do nothing, check for an existing tray icon before starting a second copy — the second instance exits by design.

SECTION 35

Troubleshooting

Symptom, likely cause, first thing to try. Start with the log; it almost always names the cause.

SYMPTOM	LIKELY CAUSE	FIX
Window opens blank or "site can't be reached"	Port 3001 in use, or backend failed to start.	Read `desktop.log`; close the other program using 3001 or change `DASHBOARD_PORT`.
SmartScreen blocks the installer	Unsigned build.	**More info** → **Run anyway**; verify the SHA-256 first.
Antivirus removes the app	Heuristic false positive on an unsigned Electron app.	Restore from quarantine and add an exclusion for the install and data folders.
Stuck on *waiting*, QR never appears	WhatsApp disabled, or the socket failed to initialise.	Check `WHATSAPP_ENABLED` and the log; restart the app.
QR scanned but status stays disconnected	Phone offline, or session rejected.	Bring the phone online; if it persists, **Log out WhatsApp** and link again.
Connected but no replies	No AI key, assistant disabled, human takeover active, or rate limited.	Check **AI Configuration** and **Test**; check the thread's AI mode; wait a minute for the limiter.

SYMPTOM	LIKELY CAUSE	FIX
Assistant says it does not know a price	Item missing, disabled, or named differently.	Add it to **Products & services** with the customer's wording in the description.
Quoted price is wrong	Catalogue price changed after the draft, or tax/delivery settings wrong.	Fix the item or **Settings** tax and delivery; ask the customer to confirm again.
Confirmation fails with "needs the customer's explicit confirmation"	Correct behaviour — no approval text.	Ask the customer to reply yes; the assistant must pass their words.
Payment stays pending after the customer paid	Webhook unreachable (loopback control port) or provider callback lost.	Press **Verify** on the payment; for automatic confirmation, expose only `/webhooks/`.
Payment shows *requires review*	Amount mismatch or ambiguous provider response.	Compare with the provider dashboard, then approve or reject deliberately.
Cannot sign in from another computer	Console bound to loopback, or Secure cookie on plain HTTP.	Set `DASHBOARD_HOST` and put HTTPS in front; set `COOKIE_SECURE=true` only with TLS.
Everyone signed out after a restart	`SESSION_SECRET` changed or was empty.	Set a stable secret in `.env`; sessions survive restarts with it.
"database is locked"	A long write transaction or an external tool holding the file.	Close other tools on the file; restart; keep backups instead of direct edits.
Installer build fails fetching Electron	Network throttling or certificate clock skew.	Use the mirror variables documented in the README and seed the Electron cache.
App crashes on launch with a missing module	Runtime dependency missing from the packaged asar.	Rebuild; the packaging step now fails early if the dependency list drifts.
Everything vanished	Different data folder, different Windows user, or a fresh install.	Check `%APPDATA%\ChatCloser`; restore a backup from `backups/`.
Harmless Node assertion on exit	Node 24 prints a libuv assertion when a fetch handle closes at shutdown.	Ignore; it does not affect data or the next start.

> **BEFORE EDITING THE DATABASE**
>
> Direct SQL on a live file can break invariants the application relies on — an order marked paid without a payment row, a reservation that never releases. Take a backup, stop the app, and prefer the API. If you must edit, change the minimum and record why.

SECTION 36

Limitations and non-goals

What this release deliberately does not do, so expectations are set before a customer finds out the hard way.

Channel

- **Unofficial WhatsApp transport.** No template approval, no green tick, no deliverability guarantee, and a real risk of restriction for bulk sending.
- **Text only.** Images, voice notes, documents and stickers are stored by WhatsApp but not processed or answered by the assistant.
- **Groups unsupported.** Keep the business number out of group chats.
- **No compose box.** Staff reply from WhatsApp after a takeover; the console triages, it does not chat.
- **One number per installation.** Linking elsewhere replaces the session.

Platform

- **One workspace per installation.** The schema is tenant-scoped, but the desktop and single-process server serve one business.
- **No sync between machines.** Two PCs are two databases; move the folder or use one server.
- **Windows-only installers.** macOS and Linux run in server mode; desktop packages must be built on those platforms.
- **No auto-update.** Upgrades are a new installer or `npm ci` plus a restart.
- **No mobile app.** The console is responsive but designed for a desktop browser.

Business functions

- **Not accounting software.** No ledgers, no VAT returns, no bank reconciliation. The accounting summary is a management view.
- **No email or SMS channel.** WhatsApp only, in this release.
- **Simple forecasting.** A moving average, not a seasonal model; it refuses to guess without fourteen days of history.
- **No multi-currency conversion.** Records keep the currency they were created in; there are no exchange rates.
- **No bulk broadcast tool.** Deliberate: it is the fastest route to a restricted number.
- **No document designer.** Quotations and invoices use a fixed, clean layout.

> THE HONEST SUMMARY
>
> ChatCloser is excellent at one thing: turning a WhatsApp conversation into a correct, confirmed, paid order with a human able to intervene. It is not a general ERP, a marketing platform or an official WhatsApp integration, and it does not pretend to be.

SECTION 37

Upgrading from ChatCloser 1.x

Your data comes with you. The upgrade adds tables and settings; it does not rewrite your customers, orders or messages.

What changed in 2.0

AREA	CHANGE
Orders	Drafts, explicit customer confirmation, an eleven-state lifecycle, stock reservations and idempotent confirmation.
Payments	Provider verification, encrypted credentials, event idempotency, manual approval, reconciliation and refunds.
Assistant	Seventeen schema-validated tools, language detection, knowledge retrieval, escalation policy and reply validation.
Console	Nineteen pages with server-side permissions, audit trail, backups, reports and a JSON API.
Desktop	Signed-format Windows installer and portable build, tray behaviour, first-run provisioning.
Security	CSRF on every form, CSP with nonces, scrypt passwords, encrypted secrets, loopback-by-default listeners.

Upgrade path

1. **Back up.** Copy the whole data folder — database, `auth_info`, `secrets.json`.
2. **Stop 1.x.** Quit the old application completely.
3. **Install 2.0.** Run the new installer. It installs alongside; the data folder is shared.
4. **Start once.** Migrations run automatically. The owner account and WhatsApp session are reused, so no QR scan is needed.
5. **Review settings.** Confirm currency, tax, delivery and assistant persona; 2.0 adds fields that default sensibly but deserve a look.
6. **Import legacy knowledge.** If 1.x used the key/value knowledge base, import it from **Settings**; it stays available as a fallback.
7. **Test.** Run the six checks in §27 before real traffic.

Compatibility notes

- Existing orders keep their numbers and totals; they simply gain the new state vocabulary.
- The legacy `close_sale` tool remains for 1.x-style flows; new conversations should use drafts and confirmation.
- Old payment rows remain readable; only new payments use verified provider lookups.
- If a 1.x install used a plaintext secret, 2.0 generates encrypted storage on first start; re-enter provider keys in **Integrations**.

SECTION 38

Glossary

The words this document and the interface use, defined once.

TERM	MEANING
Workspace	One business's data: customers, catalogue, orders, settings, team. Stored under one `business_id`.

TERM	MEANING
Business	The tenant record. The console derives it from the session; it is never taken from a request parameter.
Assistant	The AI layer: engine, prompt, tools and provider adapter.
Draft	A priced order summary that is not an order and has no financial effect.
Confirmation	The customer's explicit agreement to a draft's exact items, quantities and total.
Verified payment	A payment whose status came from a provider lookup or signature-checked event.
Manual payment	Cash or bank transfer approved by a staff member, with the approver recorded.
Reservation	Stock held for a confirmed order but not yet dispatched.
Takeover	Switching a thread to human mode so the assistant stops replying.
Escalation	Automatic handover triggered by a keyword or the model's judgement.
Knowledge chunk	A paragraph-sized piece of a knowledge document, indexed for retrieval.
Workflow	A stored trigger, conditions and actions; data, not code.
Approval	A queued human decision that gates a workflow or tool action.
Tool	A named backend operation the assistant may request, with a JSON schema.
Role	A named bundle of permissions: owner, manager, staff, accountant, viewer.
Override	A per-user permission granted or revoked on top of the role.
Audit entry	An immutable record of a privileged action with actor, action and details.
Snapshot	A consistent copy of the database taken with `VACUUM INTO`.
Encryption key	The AES-256-GCM key protecting provider credentials and backups.
Loopback	`127.0.0.1` — reachable only from the machine itself.

SECTION 39

Support, licence and credits

How to get help, what the licence permits, and what ChatCloser is built on.

Getting help

1. **Check the log first.** `desktop.log` for desktop installs, `logs/` for server installs. It names the failing component in almost every case.
2. **Reproduce with the smoke path.** A single order through sandbox, or one test message from a personal phone, narrows the problem to a component.
3. **Collect context.** Version, platform, the exact error text, and the audit entries around the failure — never paste API keys or `secrets.json`.
4. **Security issues.** Report privately to the publisher; do not open a public issue with an exploit.

Licence

ChatCloser is released under the MIT licence: use, copy, modify, merge, publish, distribute, sublicense and sell, provided the copyright notice and permission notice are included. The software is provided “as is”, without warranty of any kind. You remain responsible for

the data you process, the messages you send and compliance with WhatsApp's terms and local law.

Built with

COMPONENT	ROLE
Node.js	Runtime, built-in SQLite, HTTP and crypto.
Express	Console and JSON API.
Baileys	WhatsApp Web protocol client.
Electron	Desktop shell and installers.
SQLite	Single-file database with WAL and full-text search.
Provider SDKs / APIs	Stripe, eSewa, Khalti, and the eight model providers.

> A CLOSING NOTE
>
> The value of this system is not that it answers messages. It is that it answers them the same way every time, never invents a price, never calls an unpaid order paid, and leaves a trail when something goes wrong. Keep the catalogue honest and the knowledge base short, and it will behave like your best salesperson on their best day.

ChatCloser

BY THIRTEEN

Documentation, feature reference and operating guide for ChatCloser 2.0.0. Edition 1.0, October 2026. Prepared for operators, administrators and developers.

ChatCloser 2.0.0 · MIT licence · Windows desktop and server editions · 129 automated tests